

SRA INTERN PRESENTATION

Building Reliable LLM Systems

Detecting Attacks · Evaluating Agents

Yanna Ding — SRA Intern

Agenda

The through-line: a detector that passes today's evaluation can still fail on data it was never evaluated on, and for prompt injection the failure can be traced, and partly recovered, in the embedding space.

- **Prompt-injection detection (the focus).** Detecting malicious prompts under extreme imbalance and under distribution shift, and what the representation shows about why detection fails. [PRELIM]
- **AutoEval.** An automatic agent-evaluation framework: a deterministic autograder and an LLM judge, kept as two separate tracks.
- **KClaw.** A 49-task agent benchmark running on a real Galaxy S24, graded automatically.
- **Order.** PID takes most of the time; AutoEval and KClaw are shorter.

A high-accuracy detector can still be unusable

At a 1000:1 base rate, a detector at 1% false-positive rate raises roughly 10 false alarms for every real attack it catches, so accuracy alone does not describe usability.

- **In production, benign prompts outnumber attacks by roughly 1000 to 1.** At that ratio, the class balance, not the reported accuracy, drives what an operator experiences.
- **At a 1% false-positive rate, 1000 benign prompts produce about 10 false alarms, against roughly 1 real attack.** The resulting precision is near 9%, so most alarms are false.
- **Each false alarm is a legitimate user blocked.** A detector optimized for accuracy alone can degrade the product for the majority who are benign.
- **This is why the operating point, not accuracy, is the number to report.** Prior detectors calibrate at 0.1 to 1% FPR and report recall there (PromptShield, MIPIAD); we report precision at 90% recall, the operating point our product team uses. [USED]

Sources: [PromptShield / PIShield, arXiv:2510.14005](#) · [MIPIAD, arXiv:2605.07269](#)

What prompt injection is

Prompt injection is a user's input that overrides the model's instructions; OWASP lists it first among LLM risks, and for a deployed assistant it appears as untrusted user input.

- **The familiar form is the jailbreak.** A user writes "ignore your previous instructions and ...", and the input countermands the developer's system prompt.
- **The reason it is hard is structural.** The system's instructions and the user's input share one context with no privilege boundary, so text in the input can act as an instruction (Liu 2024). [USED]
- **In a B2B deployment, the assistant faces untrusted users.** In December 2023, a customer prompted a Chevrolet dealership's chatbot into "agreeing" to sell a Tahoe for one dollar; it went viral and the bot was taken down.
- **When the assistant can act, the effect extends from text to actions.** An agent that can send messages, make payments, or call tools turns a successful injection into an action rather than a reply.
- **We study detection at the input.** A classifier flags a malicious prompt before it reaches the agent. Indirect injection, hidden in content the assistant reads, is related and out of scope here.

Sources: [Chevrolet chatbot incident](#), [AIID 622](#) · [OWASP LLM Top 10](#) · [Liu et al. 2024](#), [arXiv:2310.12815](#)

Two failure modes, and why a lab number hides them

Two properties of production, extreme imbalance and distribution shift, are not visible in a balanced, in-distribution benchmark.

- **Imbalance shifts what a good score means.** At 1000:1, precision at a usable FPR, not accuracy, is the quantity that matters.
- **Distribution shift is expected, not exceptional.** The attacks seen in production differ from those in training, so the input distribution moves over time.
- **A mismatched evaluation can hide both.** TESSERACT reports that malware detectors near F1 0.99 in the lab drop once a realistic goodware to malware ratio (spatial bias) and real time order (temporal drift) are enforced. [USED]
- **The same effect is visible outside security.** Zillow's home-pricing model, tuned on pre-pandemic data, did not track the market when it moved and lost about 421 million dollars in one quarter.

Sources: TESSERACT, USENIX Security 2019 · Zillow post-mortem, Stanford GSB

We treat imbalance as the setting

Because imbalance is permanent in this task, we evaluate and study the detector at the deployment ratio rather than balancing it away.

- **Attacks are rare by nature, so the imbalance does not go away.** We keep it in the evaluation instead of removing it.
- **Prior work addresses imbalance at the level of training and thresholds.** MIPIAD downsamples the attack-to-benign ratio and adds weighted cross-entropy; PromptShield-style detectors calibrate at a low false-positive rate (0.1 to 1%) and report recall at 1% FPR. [USED]
- **These do not describe what imbalance does inside the model.** They set the operating point without characterizing the decision boundary that produces it.
- **We ask a representation-level question.** Where does the boundary sit under imbalance, and what happens to inputs that fall outside the training distribution. This framing connects the two failure modes.

Sources: MIPIAD, [arXiv:2605.07269](#) · PromptShield / PIShield, [arXiv:2510.14005](#)

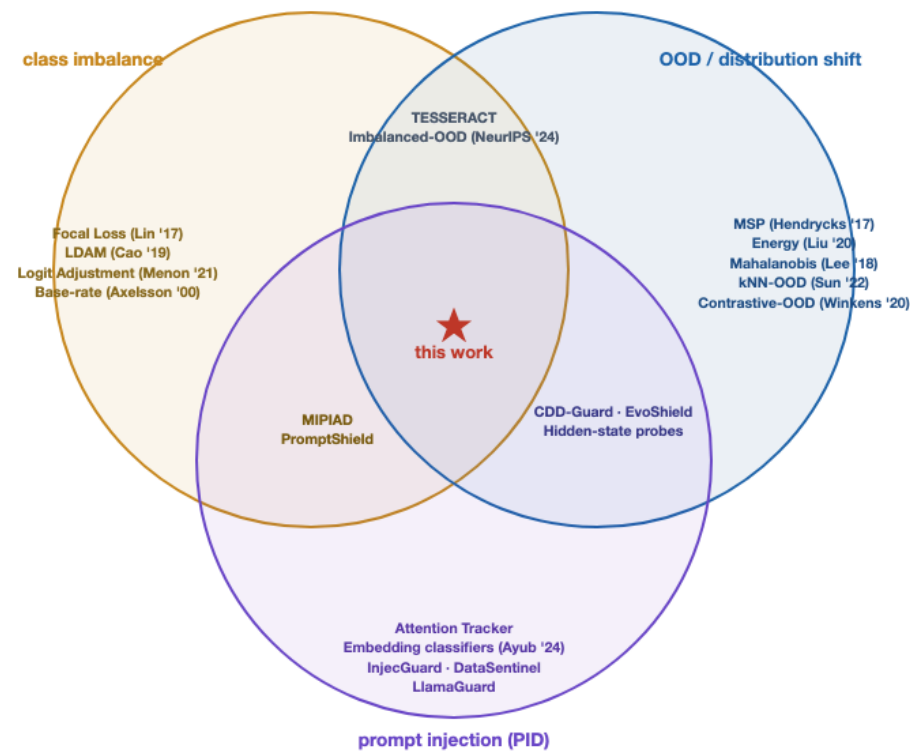
Existing detectors leave a large gap

On our data, about 60% of malicious prompts were flagged by none of the major guardrails, which is the practical reason we built a detector we control.

- **We first checked whether an off-the-shelf guardrail was sufficient.** We passed our malicious set through several market and open detectors (Armor, Bedrock Guardrails, Prompt Shields). [USED]
- **60.17% of malicious prompts were caught by none of them.** The misses did not overlap cleanly, so combining vendors did not close the gap. [MINE] [VERIFY: from an earlier market-overlap analysis; confirm which malicious set and which detectors]
- **Generic guardrails are tuned for common cases.** The rare and shifting part of the malicious class is where the gap concentrates.
- **A detector we control lets us evaluate at the real ratio and inspect its representation.** Both are needed for what follows.

Where this work sits in the problem space

The problem space, and where this work sits
Three overlapping concerns. Prior work sits in a region or a pairwise overlap; this work is at the center, prompt injection under both imbalance and distribution shift.



A benchmark engine as the backbone

The main bottleneck was training turnaround, so we built a Ray Tune benchmark engine that runs the search in parallel and lets the model family be a tuned choice.

- **The limiting factor was turnaround, not ideas.** Experiments were slow and serial, so iteration speed set the team's pace. [TEAM]
- **The engine parallelizes the search on Ray Tune.** ASHA and Hyperband stop weak trials early, directing compute to the stronger configurations. [TEAM] [USED]
- **The model family is treated as a hyperparameter.** The engine sweeps 18 checkpoints across 10 families (BERT to mmBERT, OPT, Pythia, SmolLM) under one protocol. [MINE]
- **This contribution is engineering.** It is the shared backbone the team iterates on, and it shortened the loop from design to result.

Sources: Ray Tune · ASHA, Li et al. 2020, arXiv:1810.05934

Climbing out of imbalance

A sequence of standard steps, imbalance-aware training, asymmetric loss, a stronger backbone, and more hard benign data, raised recall at a fixed low FPR from about 0.44 to 0.96 in our runs.

- **Balanced training did not hold at the real ratio.** It reached about 0.44 recall at the operating point. [MINE] [PRELIM]
- **Imbalance-aware training and asymmetric loss helped.** Training at the real ratio reached about 0.53, and asymmetric loss, which down-weights easy negatives, about 0.71 (Ridnik 2021). [USED]
- **A stronger backbone and a higher ratio continued the trend.** mmBERT-small reached about 0.76, and training at 30 to 50x about 0.82. [USED]
- **The largest single step came from data, not a loss.** About 60k additional hard benign examples raised recall to about 0.96 at a 0.0002 FPR. This data step is the PoC5 to PoC6 change that the OOD analysis starts from. [MINE]

Sources: [Asymmetric Loss](#), [Ridnik 2021](#), [arXiv:2009.14119](#) · mmBERT, jhu-clsp

Gray list: a filter we tried and moved past

A keyword hit-rate filter reduced the ratio the classifier faced, but its false negatives made it unsuitable as a standalone gate, which pushed us toward representation-based methods.

- **The idea was a cheap first-stage filter.** A hit-rate score (keywords matched over prompt length) with a threshold screens obvious cases. [MINE]
- **It reduced the imbalance reaching the classifier.** Fewer clearly-benign prompts pass through. [PRELIM]
- **Its false negatives were the problem.** **About 1.6%** of malicious prompts have little keyword overlap, so a keyword gate misses them entirely.
- **This is a limitation, and it is informative.** It is part of why we moved to methods that read intent from the representation rather than surface tokens.

Unseen attacks are the normal case

A deployed detector will meet attacks unlike those in its training set, so behavior on out-of-distribution attacks is part of the specification, not an edge case.

- **Attackers are not limited to known phrasings.** New families appear after deployment, so some fraction of production attacks is out of distribution by construction.
- **In-distribution accuracy does not predict this.** A detector can score well on held-in attacks and still miss shifted ones.
- **We study this with two real snapshots.** Two of our data iterations, which we refer to as PoC5 and PoC6, differ enough to serve as an in-distribution and an out-of-distribution set.

A model that passed its evaluation can still be the fragile one

PoC5 met its own evaluation, yet a later snapshot exposed attacks it did not cover, which suggests any currently-deployed model may be one shift away from a similar gap.

- **PoC5 was a reasonable model at the time.** It passed the evaluation it was measured against.
- **A later snapshot, PoC6, contained attacks PoC5 did not cover.** The gap was not visible from PoC5's own evaluation. [PRELIM]
- **This generalizes with appropriate caution.** A model that looks adequate today can have an analogous blind spot that only a future distribution reveals.

When the distribution shifts, performance drops

On a 2x2 cross-evaluation, the PoC5 model's AUPRC falls from 0.95 in-distribution to 0.43 on the shifted set, while the malicious sets share under 10% of their text.

- **We cross-evaluate both models on both test sets.** This separates a change in the model from a change in the data. [MINE]
- **The old model degrades on the new data.** AUPRC 0.95 in-distribution versus 0.43 out-of-distribution, in our setup. [PRELIM]
- **The two malicious sets overlap little in surface form.** About 9.8% shared text, so the shift is not a rephrasing of the same attacks.
- **The gap is specific, not global.** The new model reaches 0.98 on the new set, so the shift is learnable; the old model simply had not seen it.

Where the out-of-distribution attacks sit

In the representation, the out-of-distribution malicious set is still an approximately Gaussian cluster; its center is shifted away from the in-distribution malicious region rather than scattered.

- **We measure Mahalanobis distance to the in-distribution malicious region.** This asks how far, in standardized units, a point is from the known-malicious cluster (Lee et al. 2018). [USED]
- **The out-of-distribution malicious set is shifted, not diffuse.** Its median distance is larger (about 26 versus 19 for in-distribution), with an approximately Gaussian shape. [MINE] [PRELIM]
- **A related training-free method uses the same geometry.** Recent work scores policy violations by whitening activations and measuring distance, which is Mahalanobis distance in effect. [USED]
- **This suggests a specific failure, not noise.** The attacks are recognizable in kind but placed outside the region the boundary covers.

Sources: Mahalanobis OOD, Lee et al. 2018, arXiv:1807.03888 · Activation-space whitening, arXiv:2512.03994

The shift is asymmetric

The newer model's malicious region appears to contain the older one, but not the reverse, which is consistent with the newer data being a superset in representation space.

- **We test both directions.** Fit the reference on one model's malicious set, score the other. [MINE]
- **Forward and reverse are not symmetric.** The old attacks look in-distribution to the new model, but the new attacks look out-of-distribution to the old model (separation AUROC about 0.68 versus 0.42). [PRELIM]
- **Class separability also differs between the two representations.** Silhouette scores differ markedly (about 0.57 versus 0.13 in our measurement). [PRELIM]
- **Read together, this points to coverage.** The newer model has seen a broader region; the direction of the gap matches that.

If the geometry is the problem, reshape the geometry

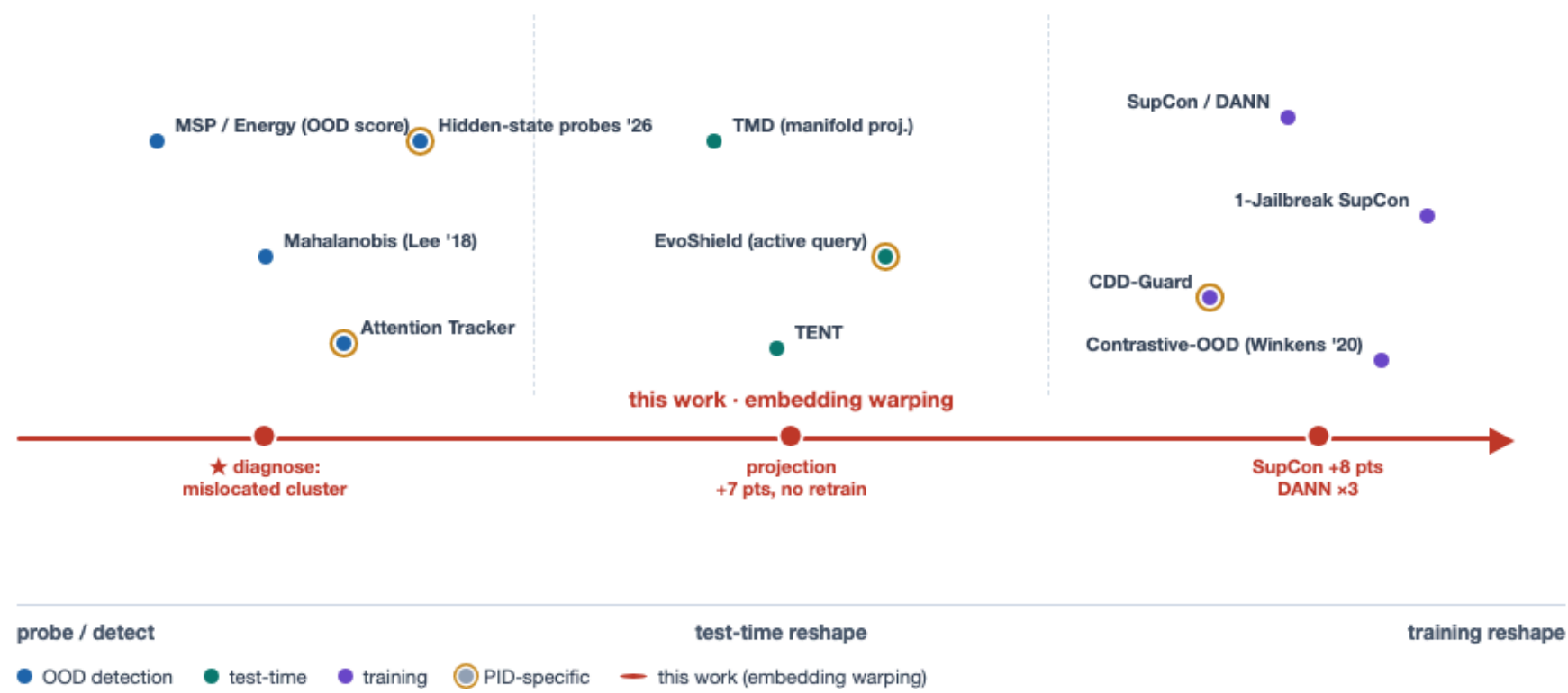
Because the failure looks like a mislocated cluster, we test whether relocating out-of-distribution inputs toward the training region recovers detection, rather than retraining on more data.

- **The hypothesis is geometric.** If out-of-distribution attacks are a shifted cluster, moving them toward the in-distribution region should let the existing boundary apply. [TEAM]
- **We call this one idea embedding warping.** Test-time projection, contrastive training, and domain-adversarial training are three ways to warp the same space toward the training region; they are instances of one hypothesis, not separate tricks. [MINE]
- **We separate two regimes.** Methods that change nothing at training time (test-time), and methods that change how the space is learned (training-based).

Methods for OOD: one line across the method axis

Methods for out-of-distribution attacks (within prompt-injection detection)

The method axis runs from probing to test-time to training. Prior work sits at one stage; this work is one geometric account, embedding warping, that runs across all three, and the red line is ours, not a chain of theirs.



Related work: what is different here

The methods on the map are known tools we use as baselines; what we did not find is any of them threaded into one geometric account for prompt injection under this imbalance.

- **Representation-space OOD detection is established, but it detects rather than relocates.** Mahalanobis distance (Lee 2018) and activation-space whitening (2512.03994) share our geometry; they score, they do not move the input. [USED]
- **The closest prompt-injection neighbors each sit in one cell.** CDD-Guard is a training-free contrastive axis; EvoShield adapts at test time by querying an external model. Ours relocates the representation with no retraining and no queries. [USED]
- **The tools we use are standard.** Contrastive and domain-adversarial training are baselines; we did not find them trained for prompt injection under held-out attack families, and no prior work threads DANN into this setting. [PRELIM]
- **The contribution is the through-line, not the tools.** One diagnosis, a mislocated cluster, and one fix, embedding warping, that holds at any intervention point, which is why several independent methods agree.

Sources: [CDD-Guard](#) · [EvoShield](#) · [TMD](#), [arXiv:2211.02878](#) · [Whitening](#), [arXiv:2512.03994](#)

Test-time methods, no retraining

Projecting representations onto the malicious manifold (top-3 directions) raised out-of-distribution AUPRC from about 0.69 to 0.76 with a 0.9% in-distribution cost; entropy-based adaptation did not help.

- **Projection onto the top directions of the malicious region.** The effective rank of that region is about 3.5, so we keep 3 directions. [MINE]
- **Out-of-distribution AUPRC improves from 0.6896 to 0.7625, about +7 points.** In-distribution drops only about 0.9%. Projecting onto the benign region does not help. [PRELIM]
- **Test-time entropy minimization (TENT) did not help here.** The gain was under half a point, consistent with an already-confident model. [USED]
- **Scope, stated plainly.** This is a proof of concept on a subsampled set; the point is that the projection changes the result, which supports the geometric reading.

Sources: TENT, Wang et al. 2021, arXiv:2006.10726 · Manifold projection defense (TMD), arXiv:2211.02878

Training-based methods

Supervised contrastive training gave the largest in-distribution-only gain (about +8 points out-of-distribution over fine-tuning); domain-adversarial training helped less, and combined best with projection.

- **Baseline is standard fine-tuning.** About 0.68 out-of-distribution AUPRC. [MINE] [PRELIM]
- **Supervised contrastive training helps most among training-time methods.** About 0.77 out-of-distribution, roughly +8 points over fine-tuning; combining it with projection reaches about 0.78. [USED]
- **Domain-adversarial training helps less on its own.** About 0.73; we designed three variants (by source, by augmentation, by cluster), two of which are still running. [TEAM] [MINE] [PRELIM]
- **A ceiling for reference.** Training with the shifted data present reaches much higher, but uses labels we would not have at deployment, so we report it only as an upper bound.

Sources: SupCon, Khosla et al. 2020, arXiv:2004.11362 · DANN, Ganin et al. 2016, arXiv:1505.07818

Results together, with the limits stated

Across methods, reshaping the representation recovers part of the out-of-distribution gap; the numbers are preliminary and several checks are still open.

- **Summary.** Fine-tuning 0.68, projection 0.76, contrastive 0.77, contrastive with projection 0.78 out-of-distribution AUPRC. [MINE] [PRELIM]
- **The same account gives a practical menu.** A cheap test-time projection with no retraining, or a trained fix with contrastive learning, depending on how much cost you can spend. [MINE]
- **We ruled out simpler explanations.** Calibration error is small (ECE about 0.001 to 0.003), there is no catastrophic forgetting, and the truly ambiguous benign fraction is tiny (about 0.01%). [PRELIM]
- **Known limitations, stated plainly.** A 20k benign subsample inflates AUPRC (full-test re-evaluation pending), and two domain-adversarial variants are unfinished.
- **What we can say.** The consistent direction across independent methods supports the geometric account, more than any single number does.

Imbalance makes the OOD failure worse

Class imbalance and distribution shift compound: recent analysis shows imbalance biases OOD detection, and in our setup the operating-point cost of the shift is larger at the deployment ratio.

- **The two problems are not independent.** Recent work characterizes a class-aware bias between balanced and imbalanced OOD detection. [USED]
- **The failure modes are specific.** Tail-class inputs tend to be flagged as OOD, and OOD inputs tend to be absorbed by head classes.
- **This matches our operating-point view.** At the deployment ratio, the precision cost of the shift is larger than a balanced number would suggest. [PRELIM]

Sources: [Rethinking OOD Detection on Imbalanced Data](#), NeurIPS 2024

Seeing it: representation and boundary

A projection of the representation, with the decision region, shows the in-distribution and out-of-distribution malicious clusters and where the boundary falls between them.

Figure placeholder: 2D projection colored by in-distribution vs out-of-distribution, with the malicious clusters and the boundary. To be generated from the stored embeddings.

- **The intent is one intuitive picture.** The diagnostics above are distances and curves; this slide is the geometry they describe. [MINE]
- **Paired with the Mahalanobis plot.** The sketch gives the intuition, the real plot gives the evidence.

What we found

In this setting, imbalance and shift are the two operative pressures, the OOD failure looks like a mislocated cluster, and reshaping the representation recovers part of it.

- **Imbalance is the setting, and it shapes the operating point.**
- **The OOD failure is structured.** A shifted, approximately Gaussian cluster, not noise. [PRELIM]
- **Reshaping the representation helps, at test time and in training.** Consistent direction across methods, preliminary in magnitude.

Why it may matter beyond this detector

The pattern, evaluate at the real setting and inspect the representation, is not specific to prompt injection and may apply to other production detectors under shift.

- **The method is transferable.** Cross-evaluation to separate model from data, and distance in representation space to locate a shift.
- **The caution is transferable too.** A passing in-distribution number does not certify behavior under shift.
- **Open directions.** A full-test re-evaluation, the remaining domain-adversarial variants, and a proper OOD-detection comparison.

Why agent evaluation is hard

Agents report success far more often than they achieve it, so evaluation cannot rely on the agent's own report.

- **Self-reported success overstates real success.** On public benchmarks the gap is large (GAIA about 92 reported versus 15 real, WebArena about 78 versus 14). [USED]
- **Task success is not a single signal.** It depends on tool calls, side effects, and constraints, not just the final message.
- **We separate two evaluation tracks.** A deterministic autograder and an LLM judge, kept apart on purpose.

The evaluation pipeline

Two independent tracks: an autograder that checks concrete effects, and an LLM judge that assesses the trajectory; they are not chained.

- **Track A, autograder.** Deterministic checks on the transcript. [MINE]
- **Track B, LLM judge.** A staged assessment of the episode. [MINE]
- **They are kept separate.** The judge does not see the autograder result, so a weak judge cannot be propped up by it.

The autograder

Per task, a small program walks the transcript's tool calls and checks concrete conditions, passing only if the observed effects meet the task's criteria.

- **Per-task grading logic.** Each task defines what a correct execution looks like. [MINE]
- **It reads the tool calls, not the prose.** Matching on tool names and arguments, and on side effects where checkable.
- **It is the harder track to get right.** Deterministic checks are unforgiving, which is the point.

The LLM judge

A staged pipeline types the task, normalizes the trajectory, evaluates against a plan, and produces a scored verdict with explicit deductions.

- **A multi-stage assessment.** Task typing, trajectory normalization, evaluation, and presentation. [MINE]
- **It produces a score with reasons.** For example, a business-travel task scored about 0.82 with listed deductions.
- **It is useful but not sufficient alone.** It complements the autograder rather than replacing it.

What AutoEval taught us

The LLM judge cannot stand alone, and the deterministic autograder is the part that is hard and worth the effort.

- **The judge alone is not reliable enough.** It needs the autograder as an independent check.
- **The autograder is where the difficulty is.** Encoding correct side effects per task is the real work.
- **Independence is the design choice that holds it together.**

KClaw: agents on a real device

A 49-task benchmark that runs on a physical Galaxy S24 and is graded automatically, spanning three difficulty tiers.

- **Real device, real actions.** Tasks execute on hardware, not a simulator. [MINE]
- **49 tasks across three tiers** (about 22 / 21 / 6). [PRELIM]
- **Graded by the autograder.** Effects are checked on the device state.

KClaw results

On the 49 tasks, the agent passed about 44, roughly 90%, under the deterministic autograder.

- **About 89.8% pass rate (44 of 49).** [PRELIM] [VERIFY]
- **Graded deterministically.** These numbers use the autograder, without an LLM judge on this set.
- **Failures cluster in the harder tier.** The tier structure shows where the agent breaks down.

What KClaw taught us

Grading real-device side effects is the binding constraint, and it is where on-device agent evaluation should invest.

- **On-device grading is the hard part.** Verifying that an action really happened is more work than running the agent.
- **The deterministic checks transfer from AutoEval.** The same discipline applies.
- **Scope is honest.** These are proof-of-concept numbers on a fixed task set. [PRELIM]

Thank you

Thank you.

Thank you for your tremendous support and inspiration.

To the VP, the managers, my mentor, and everyone I share lunch and ideas with.

References

Sources.

- Prompt injection and detection: Liu et al. 2024 (arXiv:2310.12815), Perez and Ribeiro 2022 (arXiv:2211.09527), PromptShield / PIShield (arXiv:2510.14005), OWASP LLM Top 10.
- Imbalance and evaluation: MIPIAD (arXiv:2605.07269), TESSERACT (USENIX 2019), Ridnik et al. 2021 (arXiv:2009.14119).
- OOD and representation: Lee et al. 2018 (arXiv:1807.03888), activation-space whitening (arXiv:2512.03994), Rethinking OOD on Imbalanced Data (NeurIPS 2024), TMD (arXiv:2211.02878).
- Methods: SupCon (arXiv:2004.11362), DANN (arXiv:1505.07818), TENT (arXiv:2006.10726), ASHA (arXiv:1810.05934).
- Real-world: Chevrolet chatbot incident (AIID 622), Zillow post-mortem (Stanford GSB).